

ESERCIZI

d) (4) Gestore dell'affidabilità

Si supponga che si verifichi un guasto di sistema con perdita di contenuto della memoria centrale. Al momento del guasto il contenuto del file di LOG è il seguente:

DUMP; B(T1), B(T2), B(T3), D(T1,O1,B1), I(T2,O2,A2), U(T3,O3,B3,A3), U(T3,O4,B4,A4), CK(T1,T2,T3), B(T4), B(T5), B(T6), U(T4,O5,B5,A5), A(T6), C(T1), C(T3), U(T2,O6,B6,A6), U(T4,O7,B7,A7), C(T2), guasto

Illustrare cosa accade al riavvio del sistema nell'esecuzione della procedura di ripresa a caldo.

1. $Undo = \{T_1, T_2, T_3\}$ $Redo = \{\}$
2. $Undo = \{\cancel{T_1}, \cancel{T_2}, \cancel{T_3}, T_4, T_5, T_6\}$ $Redo = \{T_1, T_3, T_2\}$
 $Undo = \{T_4, T_5, T_6\}$ $Redo = \{T_1, T_3, T_2\}$
3. $U(T_4, O_7, B_7, A_7) \rightarrow O_7 := B_7$
 $U(T_4, O_5, B_5, A_5) \rightarrow O_5 := B_5$
4. $D(T_1, O_1, B_1) \rightarrow delete(O_1)$
 $I(T_2, O_2, A_2) \rightarrow O_2 := A_2$
 $U(T_3, O_4, B_4, A_4) \rightarrow O_4 = A_4$
 $U(T_2, O_6, B_6, A_6) \rightarrow O_6 = A_6$

f) Ottimizzazione

Si consideri il seguente schema relazionale contenente i risultati di elezioni :

COLLEGIO(Codice, Provincia, Regione) **VOTO**(Candidato, Collegio, NumeroVoti)

CANDIDATO(Codice, Nome, Cognome, Lista)

Vincoli d'integrità referenziale: VOTO.Candidato $\rightarrow$ CANDIDATO, VOTO.Collegio $\rightarrow$ COLLEGIO

Data la seguente interrogazione:

```
SELECT V.NumeroVoti, C.Codice, C.Provincia
FROM COLLEGIO C JOIN VOTO V ON (C.Codice = V.Collegio)
WHERE C.Regione = 'Veneto' AND V.NumeroVoti > 1000
```

(3) Calcolare il costo dell'interrogazione in termini di numero di accessi a memoria secondaria sotto le seguenti ipotesi:

- la selezione sulla tabella VOTO eseguita attraverso una scansione della tabella medesima, il risultato della selezione viene salvato in memoria secondaria in 150 pagine (52500 righe).
- la selezione sulla tabella COLLEGIO viene eseguita attraverso una scansione della tabella medesima, il risultato della selezione viene mantenuto nel buffer.
- l'ordine di esecuzione del join è COLLEGIO $\bowtie$ VOTO
- le operazioni di join vengono eseguite con la tecnica "Nested Loop Join" con una pagina di buffer disponibile per ogni tabella
- NP(CANDIDATO) = 10, NP(VOTO) = 750, NP(COLLEGIO) = 150
- NR(CANDIDATO) = 170, NR(VOTO) = 263500, NR(COLLEGIO) = 1500,
- VAL(Regione, COLLEGIO) = 20, VAL(Collegio, VOTO) = 1500

1. Selezione su collegio

Costo = lettura + scrittura

= NP(Collegio) + 10 $\rightarrow$ mantenuto nel buffer

$$= 150$$

2. Selezione su voto

$$\text{Costo} = \text{lettura} + \text{scrittura}$$

$$= NP(\text{Voto}) + 150$$

$$= 750 + 150 = 900$$

3. Collegio X voto

$$\text{Costo} = NP(\text{selezione}(\text{collegio})) + NR(\text{selezione}(\text{collegio})) \cdot NP(\text{selezione}(\text{voto}))$$

$$= \underset{\text{(già nel buffer)}}{0} + \frac{NR(\text{collegio})}{Val(\text{regione}, \text{collegio})} \cdot 150$$

$$= \frac{1500}{20} \cdot 150$$

$$= 11250$$

$$4. \text{Costo}_{Tot} = 1. + 2. + 3. = 150 + 900 + 11250 = 1050 + 11250 = 12300$$

(2) Come cambia il costo se è disponibile un indice B*-tree sull'attributo Collegio della tabella VOTO con profondità pari a 2 pagine di memoria secondaria.

Cambia solo il costo del join

$$\begin{aligned} 3'. \text{Costo} &= NP(\text{selezione}(\text{collegio})) + NR(\text{selezione}(\text{collegio})) \cdot \left(\text{Profondità} + \frac{NR(\text{selezione}(\text{voto}))}{Val(\text{collegio}, \text{voto})} \right) \\ &= 0 + \frac{1500}{20} \cdot \left(2 + \frac{52500}{1500} \right) \\ &= \frac{15}{2} \cdot \left(2 + \frac{525}{15} \right) \\ &= 15 + \frac{525}{2} \\ &= 278 \end{aligned}$$

$$4'. \text{Costo}_{Tot} = 1. + 2. + 3' = 150 + 900 + 278 = 1050 + 278 = 1328$$

e) Esecuzione concorrente

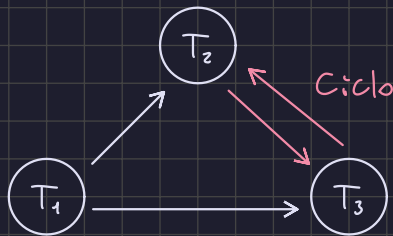
Dato il seguente schedule S:

S: $r_1(t), w_2(z), r_1(x), r_3(z), r_1(y), w_2(t), r_3(y), w_3(x), w_2(y), r_3(t), r_2(x)$

- (2) Indicare se S è view-serializzabile (VSR) oppure no (giustificare la risposta)
- (2) Indicare se S è conflict-serializzabile (CSR) oppure no (giustificare la risposta)
- (2) Indicare se S è 2PL (vale a dire, rispetta la regola di serializzabilità del locking a due fasi)

• CSR

Conflicts: $\{ (r_1(x), w_3(x)), (w_3(x), r_2(x)), (r_1(y), w_2(y)), (r_3(y), w_2(y)), (w_2(z), r_3(z)), (r_1(t), w_2(t)), (w_2(t), r_3(t)) \}$



Grafo aciclico, quindi non è CSR e neanche 2PL

S: $r_1(t), w_2(z), r_1(x), r_3(z), r_1(y), w_2(t), r_3(y), w_3(x), w_2(y), r_3(t), r_2(x)$

• VSR

Legge-do = $\{ (r_2(x), w_3(x)), (r_3(z), w_2(z)), (r_3(t), w_2(t)) \}$

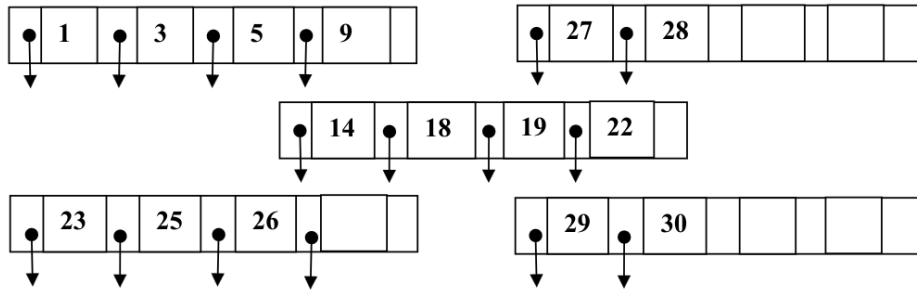
Scritture-finali = $\{ w_3(x), w_2(y), w_2(z), w_2(t) \}$

Non è neanche VSR

g) B+-tree

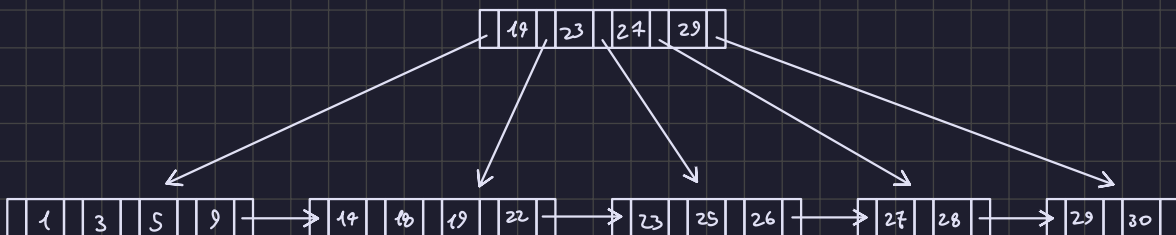
Data la seguente lista di possibili valori chiave $L = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30)$:

a) (1) costruire un B+-tree (fan-out=5) che contenga i seguenti nodi foglia:

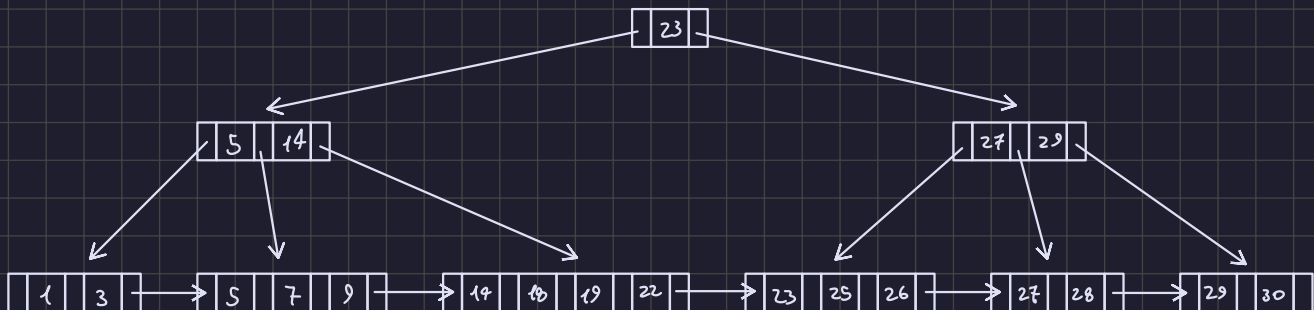


b) (2) mostrare l'albero dopo l'inserimento del valore chiave 7

Fan-out=5 $\rightarrow$ $2 \leq \# \text{chiave} \leq 4$
 $3 \leq \# \text{puntatori} \leq 5$



Inserimento di 7



c) (2) partendo dal risultato del punto precedente mostrare l'albero dopo la rimozione del valore chiave 27.

